

总结篇 | mattpocock/skills 全系列回顾与速查合集

原创 共渡时艰 共渡时艰 2026年8月19日 08:05 广东



mattpocock

3.5万人看过 32条内容



前言

从「mattpocock/skills 是什么」讲到「用完整流水线开发 EShop 的优惠券叠加功能」，从零到一，我们把这条链路从头到尾走了一遍。安装、设计灵魂、共享语言、需求访谈、规划落地、反馈回路、实战整合。

这篇文章不是新的一课。它是整趟旅程的地图，也是你以后每天会用到的速查手册。课程索引、技能速查、触发模型、术语、学习路径、EShop 演进，全部钉在一起，方便你随时回来翻查。

全系列课程索引

基础篇（01-03）

01 - mattpocock/skills 完全入门指南 定义（Skill 是什么/不是什么）、四大失败模式、两种安装哲学、setup。→ 完全没接触过的读者

02 - 理解 mattpocock/skills 设计灵魂 四坑 → 解药对照、user-invoked/model-invoked 触发模型、积木 vs 框架。→ 想理解为什么这么设计的读者

03 - 为什么 mattpocock/skills 让你别再调 prompt，而是给 AI 建领域词典 共享语言、术语表、模块地图、三条建法、语义扩散。→ 嫌 AI 啰嗦、想统一项目术语的读者

实战篇（04-07）

04 - mattpocock/skills 为啥不让我直接写代码？grill-me 让你怀疑人生 grill-me / grill-with-docs / grilling 三兄弟、决策树访谈、访谈 vs 文档权衡。→ 需求老是被 AI 理解偏的读者

05 - mattpocock/skills 聊完需求 AI 就失忆？一条流水线把对话变成代码 对话 → 规格 → 工单 → 代码，wayfinder 定位、规格不是蓝图。→ 想让 AI 开发流程化、可审查的读者

06 - mattpocock/skills AI 写代码是闭眼飞？tdd 加 diagnosing-bugs 让它睁开眼 红绿重构、六步排错、垂直切片、反覆盖率指标。→ 被 AI 的坏代码折磨的读者

07 - mattpocock/skills 零件都齐了，装成机器才是真本事 八环节完整流程实战、渐进引入。→ 想看到所有知识点整合效果的读者

专题

wayfinder 专题 | mattpocock/skills AI 装不下的大工程？把迷雾画成地图，一次探一步 全程手把手，gh 命令建图走图、四类决策工单逐个解析、EShop Java 代码示例。→ 接手一个会话装不下的大工程、路径不明的读者

loop-me 专题 | mattpocock/skills 你把每天重复的事情写成说明书交给 AI，一次说清，永久代劳 loop 视角、Trigger / Checkpoint / Push right / Brief 四词拆解、workflow spec 示范、grilling 实战。→ 想把生活里重复的杂事委托给 AI 的读者

batch-grill-me 专题 | mattlepcock/skills grill-me 别挤牙膏：能问的问题一轮问完，答完自动解锁下一批 设计树、前沿、轮次三件套拆解，批量访谈与一次一问的取舍，EShop 会员等级三轮实战。→ 嫌一次一个问题太磨叽、想批量过需求的读者

teach 专题 | Mattlepcock Skills 新知识学完就忘记了？mattlepcock/skills 来助力 工作区七件套、Knowledge / Skills / Wisdom、流利 vs 存储、最近发展区、/teach 五步实战。→ 想系统学一个新主题、不想学完就忘的读者

improve-codebase-architecture 专题 | Mattlepcock Skills 代码写久了为什么变浆糊？让 AI 给代码做一次体检 深/浅模块词汇表、Explore 找热点 → HTML 报告摆候选 → grilling 聊到底三阶段、EShop 订单管线实战与完整决策树访谈。→ 看着代码库觉得哪儿不对但说不上来的读者

技能速查

安装与初始化

```
1 # 方式一: Claude Code 插件 (只读捆绑, 自动更新)
2 claude plugins install mattlepcock-skills
3 # 方式二: skills.sh (可编辑文件, 完全自控)
4 npx skills@latest add mattlepcock/skills
5 # 二选一, 不要都装
6 # 每个仓库初始化一次
7 /setup-matt-pocock-skills
```

按失败模式速查

AI没有理解你的意思？

04 - mattlepcock/skills 为啥不让我直接写代码？grill-me 让你怀疑人生

```
1 /grill-me
2 # 或者
3 /grill-with-docs
```

AI太啰嗦？

03 - 为什么 mattlepcock/skills 让你别再调 prompt，而是给 AI 建领域词典

```
1 # 创建 CONTEXT.md
2 /domain-modeling
```

代码有问题？

06 - mattlepcock/skills AI 写代码是闭眼飞？tdd 加 diagnosing-bugs 让它睁开眼

```
1 /tdd
2 /diagnosing-bugs
```

时间久了，架构变混乱？

improve-codebase-architecture 专题 | mattlepcock/skills 代码写久了为什么变浆糊？

```
1 /improve-codebase-architecture
```

不知道如何使用？

07 - mattpocock/skills 零件都齐了，装成机器才是真本事

```
1 /ask-matt
```

按开发阶段速查

路由阶段

07 - mattpocock/skills 零件都齐了，装成机器才是真本事

```
1 /ask-matt
```

访谈阶段

04 - mattpocock/skills 为啥不让我直接写代码？ grill-me 让你怀疑人生

batch-grill-me 专题 | mattpocock/skills grill-me 别挤牙膏：能问的问题一轮问完，答完自动解锁下一批

```
1 /grill-me
2 /grill-with-docs
3 /batch-grill-me
```

委托阶段

loop-me 专题 | mattpocock/skills 你把每天重复的事情写成说明书交给 AI，一次说清，永久代劳

```
1 /loop-me
```

规范阶段

05 - mattpocock/skills 聊完需求 AI 就失忆？一条流水线把对话变成代码

```
1 /to-spec
```

工单阶段

05 - mattpocock/skills 聊完需求 AI 就失忆？一条流水线把对话变成代码

wayfinder 专题 | mattpocock/skills AI 装不下的大工程？把迷雾画成地图，一次探一步

```
1 /to-tickets
2 /wayfinder
```

开发阶段

05 - mattpocock/skills 聊完需求 AI 就失忆？一条流水线把对话变成代码

06 - mattpocock/skills AI 写代码是闭眼飞？ tdd 加 diagnosing-bugs 让它睁开眼

```
1 /implement
2 /tdd
```

Debug调试阶段

06 - mattpocock/skills AI 写代码是闭眼飞? tdd 加 diagnosing-bugs 让它睁开眼

```
1 /diagnosing-bugs
```

代码评审阶段

07 - mattpocock/skills 零件都齐了，装成机器才是真本事

```
1 /code-review
```

完整流程一条线

```
1 /ask-matt
2 /grill-me
3 /to-spec
4 /to-tickets
5 /implement # 内嵌 tdd
6 /diagnosing-bugs
7 /code-review
```

全程由 CONTEXT.md（共享语言）支撑。

触发模型速查

🚩 user-invoked（用户输入触发）

/grill-me、/to-spec、/to-tickets、/implement、/ask-matt → 负责编排流程，只有用户显式输入才触发

🚩 model-invoked（模型自动触发）

tdd、diagnosing-bugs、code-review、grilling、domain-modeling → 承载可复用纪律，模型按任务匹配自动调用

🚩 规则

user-invoked 可以调用 model-invoked，但绝不调用另一个 user-invoked

术语索引

💡 Skill

结构化 workflow 文件（SKILL.md），不是 prompt

💡 四大失败模式

需求不清、过于啰嗦、代码不工作、架构腐化（anecdotal evidence）

💡 CONTEXT.md

项目共享语言文档，治「AI 太啰嗦」

💡 User-invoked

用户输入触发的技能，负责编排

💡 Model-invoked

模型自动调用的技能，承载纪律

💡 Deep Module

大量行为藏在简单接口后的模块

💡 Red-Green-Refactor

先写失败测试 → 最小实现 → 重构

💡 Vertical Slice

一次实现一个完整可用的功能片段

💡 Shared Language

术语与代码同源的共享词典（ubiquitous language）

💡 Semantic Diffusion

语义扩散，一个概念长出多个名字，词典失效的根源

推荐学习路径

如果你是新读者

按顺序读，每课的前置知识是上一课，别跳。

如果你已经用过 AI 编码，想直接上手

1. 01 - mattlecock/skills 完全入门指南
2. 03 - 为什么 mattlecock/skills 让你别再调 prompt，而是给 AI 建领域词典
3. 04 - mattlecock/skills 为啥不让我直接写代码？grill-me 让你怀疑人生
4. 07 - mattlecock/skills 零件都齐了，装成机器才是真本事

如果你只解决一个具体问题

1. 04 - mattlecock/skills 为啥不让我直接写代码？grill-me 让你怀疑人生
2. 06 - mattlecock/skills AI 写代码是闭眼飞？tdd 加 diagnosing-bugs 让它睁开眼
3. 03 - 为什么 mattlecock/skills 让你别再调 prompt，而是给 AI 建领域词典
4. improve-codebase-architecture 专题 | mattlecock/skills 代码写久了为什么变浆糊？

最后想说的话

写到这里，照例说几句掏心窝的。

如果你只从这个系列带走三件事，我希望是这三件。

第一，先定义，再决定。不管面对什么新技术，先把它是什么、不是什么搞清楚，再谈要不要用。语义扩散是这个时代的通病，词汇越来越泛滥，定义越来越模糊。**能给出清晰定义的人，才有资格谈判断。**

第二，工具是工具，不是信仰。**mattpocock/skills 很好用，但它是可以裁剪的。觉得 tdd 有用就留 tdd，觉得全流程太重就砍掉一半。任何工具，包括这个，一旦变成「因为流程要求所以这么做」，就该瘦身了。**

第三，纪律是目的，AI 是手段。这套技能的全部价值，在于逼你做的那些事，先问清需求、先写测试、先审查。工具会过时，纪律不会。**你带走的应该是「这些事值得做」的判断，而不是「必须用这套技能」的依赖。**

到这里，主系列 8 课就结束了，记住，流程可以裁剪，先问清、先测好、先审完，不能丢。**磨平一点信息差，哪怕只是你自己和昨天的自己之间，那一点。**

我是超级奶爸老谭！我们下个系列课程见 🤝

#mattpocock #tdd #skills #AI #人工智能

喜欢作者

📁 合集索引 · 目录

上一篇

总结篇：Trellis 全系列回顾与速查合集

下一篇

总结篇：superpowers 全系列回顾与速查合集